

FACULDADE PRESIDENTE ANTÔNIO CARLOS DE
UBERLÂNDIA

Guilherme Osvaldino de Oliveira

**SISTEMA DE MONITORAMENTO DE
ÍNDICES GLICÊMICOS**

Uberlândia, Brasil

2026

FACULDADE PRESIDENTE ANTÔNIO CARLOS DE
UBERLÂNDIA

Guilherme Osvaldino de Oliveira

SISTEMA DE MONITORAMENTO DE ÍNDICES GLICÊMICOS

Trabalho final apresentado ao Curso Superior em Análise e Desenvolvimento de Sistemas da Faculdade Presidente Antônio Carlos de Uberlândia, como parte dos requisitos necessários à obtenção do título de Analista e Desenvolvedor de Sistemas.

Orientador: Prof. Fabricio Geraldo Araujo

Uberlândia, Brasil

2026

Dedico este trabalho aos meus pais, familiares e amigos que sempre me incentivaram e apoiaram ao longo desta jornada acadêmica.

Agradecimentos

Primeiramente, agradeço a Deus por me conceder força, sabedoria e determinação para superar os desafios encontrados ao longo desta jornada acadêmica.

Aos meus familiares, pelo apoio incondicional, incentivo e compreensão em todos os momentos, especialmente durante os períodos de maior dedicação aos estudos.

Aos professores do curso de Análise e Desenvolvimento de Sistemas, pelos ensinamentos compartilhados, pela dedicação e pelo compromisso com a formação profissional e pessoal dos alunos.

Ao meu orientador, Prof. Fabricio Geraldo Araujo, pela orientação, paciência, disponibilidade e pelos valiosos conhecimentos transmitidos durante o desenvolvimento deste trabalho, contribuindo significativamente para a sua realização.

Resumo

O presente trabalho apresenta o desenvolvimento de uma aplicação web voltada para o monitoramento clínico e controle de índices glicêmicos, atuando como um diário de saúde digital para pacientes diagnosticados com *Diabetes Mellitus* ou em estados pré-diabéticos. O problema identificado está relacionado à utilização de métodos manuais e obsoletos de anotações, como cadernos de papel, o que acarreta a perda de históricos, omissão de dados e severas dificuldades na consolidação de relatórios estatísticos para avaliação médica. O objetivo do projeto foi desenvolver um sistema seguro e responsivo capaz de centralizar o registro cronológico de taxas de glicose, segmentá-las por momentos biológicos e automatizar o cálculo de médias preditivas. A metodologia utilizada caracteriza-se como pesquisa aplicada, de natureza descritiva e abordagem mista, envolvendo levantamento de requisitos, modelagem relacional de dados, desenvolvimento de software e testes funcionais em ambiente de produção. Para o desenvolvimento, foram utilizadas as tecnologias Python, framework Django, SGBDR PostgreSQL e a plataforma de nuvem Render, com interface baseada na estética *Glassmorphism*. Os resultados demonstraram que o sistema atendeu plenamente aos objetivos propostos, permitindo a persistência íntegra dos exames, o cálculo em tempo real da estimativa da Hemoglobina Glicada (HbA1c) e a eliminação de erros de cabeçalho em servidores HTTP através do middleware WhiteNoise. Conclui-se que a aplicação contribui de forma significativa para a autonomia do paciente e para a eficiência do acompanhamento médico, oferecendo uma solução robusta, gratuita e multiplataforma para a gestão de dados de saúde.

Palavras-chave: sistema web. monitoramento glicêmico. diabetes. Python. Django. nuvem.

Abstract

This paper presents the development of a web application designed for clinical monitoring and glycemic index control, serving as a digital health diary for patients diagnosed with *Diabetes Mellitus* or in pre-diabetic states. The identified problem is related to the use of manual and obsolete notation methods, such as paper notebooks, which causes history loss, data omission, and severe difficulties in consolidating statistical reports for medical evaluation. The main objective of the project was to develop a secure and responsive system capable of centralizing the chronological record of glucose levels, segmenting them by biological moments, and automating the calculation of predictive averages. The methodology adopted is classified as applied research, with a descriptive nature and mixed approach, involving requirements gathering, relational database modeling, software development, and functional testing in a production environment. The technologies used in the development included Python, Django framework, PostgreSQL RDBMS, and the Render cloud platform, with an interface based on the *Glassmorphism* aesthetics. The results demonstrated that the system fully achieved the proposed objectives, allowing the complete persistence of test data, real-time calculation of Estimated Hemoglobin A1c (HbA1c), and the elimination of header errors in HTTP servers through the WhiteNoise middleware. It is concluded that the application contributes significantly to patient autonomy and medical follow-up efficiency, providing a robust, free, and multiplatform solution for health data management.

Keywords: web system. glycemic monitoring. diabetes. Python. Django. cloud.

Lista de ilustrações

Figura 1 – Diagrama Entidade-Relacionamento da Base de Dados.	20
Figura 2 – Diagrama de Casos de Uso do Sistema.	20
Figura 3 – Tela de Login.	21
Figura 4 – Dashboard.	22
Figura 5 – Tela de Inserções.	22

Lista de abreviaturas e siglas

ACID	<i>Atomicity, Consistency, Isolation, Durability</i> (Atomicidade, Consistência, Isolamento e Durabilidade)
ADAG	<i>A1C-Derived Average Glucose</i> (Glicose Média Derivada de A1C)
API	<i>Application Programming Interface</i> (Interface de Programação de Aplicação)
CI/CD	<i>Continuous Integration / Continuous Deployment</i> (Integração Contínua / Implantação Contínua)
CRUD	<i>Create, Read, Update, Delete</i> (Criação, Leitura, Atualização e Exclusão)
CSS	<i>Cascading Style Sheets</i> (Folhas de Estilo em Cascata)
DER	Diagrama Entidade-Relacionamento
HTML	<i>HyperText Markup Language</i> (Linguagem de Marcação de Hipertexto)
HTTP	<i>Hypertext Transfer Protocol</i> (Protocolo de Transferência de Hipertexto)
JSON	<i>JavaScript Object Notation</i> (Notação de Objetos JavaScript)
JWT	<i>JSON Web Token</i>
MTV	<i>Model-Template-View</i>
ORM	<i>Object-Relational Mapping</i> (Mapeamento Objeto-Relacionamento)
OWASP	<i>Open Web Application Security Project</i>
PaaS	<i>Platform as a Service</i> (Plataforma como Serviço)
SGBDR	Sistema Gerenciador de Banco de Dados Relacional
UI	<i>User Interface</i> (Interface do Usuário)
UX	<i>User Experience</i> (Experiência do Usuário)

Sumário

1	INTRODUÇÃO	10
1.1	Contextualização	10
1.2	Problema de Pesquisa	10
1.3	Objetivos	10
1.3.1	Objetivo Geral	10
1.3.2	Objetivos Específicos	11
1.4	Justificativa	11
1.5	Resultados Esperados	11
2	CONTEXTUALIZAÇÃO E INOVAÇÃO	12
2.1	Cenário Atual do Monitoramento de Saúde	12
2.2	Inovações Propostas pelo Sistema	12
3	REFERENCIAL TEÓRICO	13
3.1	Engenharia de Software e Processos Web	13
3.2	Arquitetura Cliente-Servidor e o Padrão MTV	13
3.3	Desenvolvimento Backend com Python, Django e Segurança de Dados	14
3.4	Persistência Relacional e Modelagem de Dados	14
3.5	Interface do Usuário, Responsividade e a Estética Glassmorphism	15
4	METODOLOGIA	16
4.1	Tipo de Pesquisa	16
4.2	Sujeitos do Teste e Protocolo de Usabilidade	16
4.3	Instrumentos de Pesquisa	17
4.4	Limitações Metodológicas	17
5	DESENVOLVIMENTO E RESULTADOS	19
5.1	Processo de Desenvolvimento Técnico	19
5.2	Modelagem de Sistemas (UML e Dados)	19
5.3	Apresentação das Telas do Sistema	21
5.3.1	Tela de Autenticação e Login	21
5.3.2	Painel de Controle Principal (Dashboard)	21
5.3.3	Formulário de Inserção de Novas Medições	22
5.4	Testes e Resultados	22
6	CONCLUSÃO	24

6.1	Resumo dos Resultados	24
6.2	Limitações e Trabalhos Futuros	24
	REFERÊNCIAS	26
	APÊNDICES	27
	APÊNDICE A – LOGS DE DEPURAÇÃO E ANOTAÇÕES DE PRO-	
	CESSO	28
A.1	Logs do Processo de Inicialização do Servidor de Aplicação (WSGI	
	Gunicorn) em Nuvem	28
A.2	Logs de Erro de Tipo MIME e Resolução via Middleware (F12	
	Developer Tools)	28
	ANEXOS	30
	ANEXO A – CONSIDERAÇÕES FUTURAS E IDEIAS NÃO IM-	
	PLEMENTADAS	31
A.1	Separação Arquitetural para API REST com Django REST Fra-	
	mework (DRF)	31
A.2	Implementação de Frontend Independente com Ecossistema React	31

1 Introdução

1.1 Contextualização

O avanço da tecnologia da informação aplicado à saúde digital, conceito globalmente conhecido como *e-health*, tem transformado a relação de pacientes com o gerenciamento de condições crônicas. Dentre as patologias que exigem monitoramento contínuo e rigoroso, o *Diabetes Mellitus* destaca-se pelo alto impacto na qualidade de vida da população. Conforme diretrizes da Sociedade Brasileira de Diabetes (SBD, 2024), o controle rigoroso dos níveis de glicose no sangue é o principal fator para a prevenção de complicações agudas e crônicas, tais como nefropatias, retinopatias e problemas cardiovasculares. A automação e a centralização desses registros em sistemas computacionais modernos baseados na web facilitam a rotina de coleta de dados, permitindo que o paciente compreenda suas variações glicêmicas cotidianas.

1.2 Problema de Pesquisa

Tradicionalmente, o acompanhamento ambulatorial de taxas glicêmicas depende de anotações manuais efetuadas pelos próprios pacientes em cadernos de papel ou planilhas locais desestruturadas. Esse cenário acarreta problemas operacionais crônicos: a perda frequente dos históricos físicos, a omissão involuntária de medições, a falta de padronização na indicação dos momentos biológicos (como estados de jejum ou períodos pós-prandiais) e, fundamentalmente, a impossibilidade de realizar cálculos estatísticos preditivos em tempo real. Diante disso, emerge a seguinte questão de pesquisa: **Como uma aplicação web responsiva baseada no framework Django e computação em nuvem pode otimizar o gerenciamento de dados glicêmicos, automatizando o cálculo de indicadores clínicos como a Hemoglobina Glicada Estimada (HbA1c)?**

1.3 Objetivos

1.3.1 Objetivo Geral

Desenvolver e implantar em ambiente de produção em nuvem uma aplicação web responsiva e segura para o monitoramento de índices glicêmicos, fornecendo ferramentas analíticas para o cálculo automático de médias e estimativas clínicas correlacionadas ao diabetes.

1.3.2 Objetivos Específicos

- Realizar o levantamento de requisitos funcionais e não funcionais focados na experiência do usuário e na segurança de dados de saúde;
- Modelar a estrutura do banco de dados relacional utilizando diagramas entidade-relacionamento normalizados;
- Desenvolver o backend estruturado em linguagem Python com o framework Django, empregando mecanismos nativos de proteção contra vulnerabilidades web;
- Implementar uma interface responsiva baseada em folha de estilo CSS customizada sob a estética visual *Glassmorphism*, focada no conforto visual e legibilidade;
- Configurar o deploy contínuo (CI/CD) integrado ao GitHub na plataforma Render com banco de dados PostgreSQL em nuvem.

1.4 Justificativa

A relevância deste projeto fundamenta-se na fusão entre a engenharia de software pragmática e o impacto social na saúde pública. O desenvolvimento de um software de monitoramento glicêmico gratuito e acessível elimina as barreiras financeiras e tecnológicas de aplicativos comerciais proprietários. Sob a óptica técnica, o projeto justifica-se pelo uso de tecnologias modernas e robustas (Python/Django), que garantem alta velocidade de desenvolvimento e segurança arquitetural nativa. Para o ecossistema acadêmico, o trabalho ilustra de forma prática as etapas de transição de um sistema monolítico local para uma infraestrutura escalável distribuída inteiramente em nuvem.

1.5 Resultados Esperados

Espera-se que a aplicação opere de maneira estável na internet, entregando uma plataforma onde usuários possam cadastrar suas contas de forma isolada e segura. O painel de controle principal (*Dashboard*) deve exibir em tempo real a atualização das médias aritméticas e a conversão preditiva da Hemoglobina Glicada baseada na fórmula internacional ADAG. Adicionalmente, busca-se alcançar altos níveis de satisfação nos testes de usabilidade, comprovando a eficácia e o conforto visual da interface desenvolvida para smartphones e computadores.

2 Contextualização e Inovação

2.1 Cenário Atual do Monitoramento de Saúde

O gerenciamento moderno do bem-estar individual migrou do modelo puramente reativo para o preventivo e preditivo. No entanto, no ecossistema de pacientes com diabetes no Brasil, ainda há um abismo tecnológico entre os sistemas hospitalares integrados e a rotina de autocuidado doméstico do cidadão. Aplicativos móveis disponíveis em lojas digitais frequentemente expõem os dados a propagandas invasivas, cobram taxas de assinatura abusivas ou carecem de responsividade multiplataforma fluida, forçando o retorno de muitos usuários ao controle manual em blocos de papel.

2.2 Inovações Propostas pelo Sistema

A aplicação web desenvolvida introduz diferenciais técnicos estruturados para mitigar as lacunas do mercado atual. A primeira inovação reside na sua **automação analítica nativa**: o sistema não atua como um mero repositório estático de números; ele atua de forma ativa ao interpretar os dados históricos dinamicamente através do motor do Django e fornecer ao usuário o cálculo aproximado da Hemoglobina Glicada Estimada (HbA1c). Esse indicador estatístico antecipa o cenário que o paciente encontrará em seus exames laboratoriais trimestrais de sangue venoso, empoderando-o na tomada de decisões alimentares e de rotina.

A segunda inovação concentra-se no aspecto de design e usabilidade (**UI/UX**). O software rompe com as interfaces médicas tradicionais — comumente claras, frias e monocromáticas — ao implementar de forma nativa e manual a estética *Glassmorphism* sobre um tema escuro (*Dark Mode*). Painéis translúcidos com efeitos de desfoque de fundo e bordas suaves foram aplicados estrategicamente para reduzir a fadiga visual noturna, tornando o ato repetitivo e diário de registrar a glicemia uma experiência digital menos estressante e visualmente agradável.

Por fim, a inovação em **arquitetura e infraestrutura** manifesta-se no modelo de implantação. A solução foi concebida para operar sob custo operacional zero para o microempreendedor ou desenvolvedor, utilizando o tier de computação gratuita da PaaS Render associado ao SGBDR PostgreSQL em nuvem. A resiliência do sistema é assegurada pela automação de scripts de deploy e pelo empacotamento de arquivos estáticos via middleware WhiteNoise, entregando performance e estabilidade comparáveis a plataformas comerciais de alto custo de manutenção.

3 Referencial Teórico

3.1 Engenharia de Software e Processos Web

O desenvolvimento de sistemas de informação voltados à saúde digital (*e-health*) exige a aplicação rigorosa de metodologias que confirmam previsibilidade, segurança e qualidade ao produto final. Segundo Sommerville (2019), a Engenharia de Software provê as estruturas teóricas e práticas necessárias para conduzir um projeto desde a sua concepção abstrata até a fase de manutenção operacional.

No contexto de aplicações para a Web, os ciclos de desenvolvimento demandam atenção redobrada à modularização do código, proteção de dados sensíveis e clareza das interfaces. Pressman (2016) corrobora essa visão ao destacar que a divisão estrutural de um sistema em camadas independentes simplifica a depuração de erros e viabiliza o reuso de componentes de software. A arquitetura adotada no Medidor de Glicemia reflete esses preceitos ao isolar estritamente a lógica de negócio das regras de apresentação visual, mitigando o acoplamento tecnológico por meio do desacoplamento nativo de componentes.

3.2 Arquitetura Cliente-Servidor e o Padrão MTV

A Web moderna opera fundamentalmente sobre o modelo arquitetural Cliente-Servidor. Conforme documentado pela Mozilla Web Docs (2024), esse modelo baseia-se no envio de requisições estruturadas sob o protocolo HTTP/HTTPS por parte de um cliente (o navegador web) e o subsequente processamento e resposta por um servidor remoto.

Para a sustentação do Medidor de Glicemia, essa arquitetura foi estruturada sob o padrão **MTV (Model-Template-View)** nativo do framework Django, desmembrado em três camadas lógicas interconectadas:

- **Model (Camada de Persistência):** Onde reside a definição estrutural dos dados e o mapeamento objeto-relacional (ORM). Ela faz a ponte direta com o SGBDR PostgreSQL em nuvem, abstraindo comandos SQL e garantindo a integridade dos dados de usuários e medições.
- **Template (Camada de Apresentação):** Responsável por traduzir os dados estruturados em elementos visuais inteligíveis ao usuário através de arquivos HTML5 estruturados, folhas de estilo CSS3 avançadas (estética *Glassmorphism*) e manipulações dinâmicas de interface.

- **View (Camada de Aplicação/Backend):** Onde reside o núcleo de processamento lógico do sistema em Python. A *View* intercepta as requisições HTTP enviadas pelo cliente, executa as validações necessárias (como regras de autenticação ou consistência dos limites de glicemia), interage com o *Model* para persistência ou consulta e renderiza o *Template* correspondente de resposta.

3.3 Desenvolvimento Backend com Python, Django e Segurança de Dados

A documentação oficial do Django (DJANGO SOFTWARE FOUNDATION, 2024) conceitua o ecossistema como um framework web de alto nível em Python que incentiva o desenvolvimento rápido e o design limpo e pragmático. No cenário do gerenciamento de dados de saúde, o backend atua como um controlador rígido que impede o vazamento de informações e garante o isolamento de registros por usuário autenticado, blindando a sessão de cada paciente.

Para mitigar riscos de segurança críticos descritos no topo de vulnerabilidades da OWASP (*Open Web Application Security Project*), o sistema adota as proteções nativas do ecossistema Django:

- **Prevenção contra SQL Injection:** O uso do Django ORM força a parametrização estrita de todas as consultas executadas no PostgreSQL, tratando as entradas dos usuários de forma puramente literal, impedindo a injeção de scripts maliciosos.
- **Proteção contra Cross-Site Request Forgery (CSRF):** A aplicação exige a validação obrigatória de *tokens* criptográficos efêmeros em todas as requisições do tipo POST, garantindo que os formulários de inserção de glicemia sejam submetidos exclusivamente a partir da interface oficial e por usuários autenticados.

3.4 Persistência Relacional e Modelagem de Dados

O armazenamento de informações estruturadas de saúde requer um mecanismo que garanta a consistência absoluta das transações. Date (2004) postula que os Sistemas Gerenciadores de Bancos de Dados Relacionais (SGBDR), operando sob o modelo de tabelas interconectadas, asseguram a integridade dos dados por meio de restrições de chaves primárias, estrangeiras e integridade referencial.

A escolha do PostgreSQL justifica-se por sua robustez corporativa, forte suporte a indexações complexas e total conformidade com as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade). A modelagem foi orientada pelas regras de normalização de dados, utilizando o Diagrama Entidade-Relacionamento (DER) para anular

redundâncias, otimizar consultas analíticas e garantir o vínculo unívoco entre a tabela de usuários nativa do Django (`auth_user`) e o histórico individual de exames.

3.5 Interface do Usuário, Responsividade e a Estética Glassmorphism

Duckett (2011) salienta que o uso correto e semântico do HTML5 constitui a espinha dorsal de qualquer aplicação web acessível, enquanto as folhas de estilo (CSS3) ditam a hierarquia visual. Dada a heterogeneidade dos dispositivos de acesso atuais, a responsividade tornou-se um requisito não funcional obrigatório para softwares médicos domésticos.

A interface do Medidor de Glicemia diferencia-se pela implementação manual e nativa da estética visual **Glassmorphism (Efeito Vidro)**. Baseado em propriedades avançadas de CSS3 como `backdrop-filter: blur()`, transparências controladas (`rgba`), sombras difusas (`box-shadow`) e bordas sutis, este estilo visa criar interfaces limpas, modernas e visualmente confortáveis. O objetivo de UI/UX foi reduzir a carga de estresse associada ao monitoramento de doenças crônicas, oferecendo um ambiente escuro, de alta legibilidade, otimizado para telas de smartphones e interações de toque rápidas.

4 Metodologia

4.1 Tipo de Pesquisa

O presente estudo caracteriza-se como uma pesquisa de natureza aplicada, com caráter descritivo e abordagem mista (qualitativa e quantitativa). É classificada como aplicada pois objetiva a geração de conhecimentos direcionados à solução de um problema prático e latente: o desenvolvimento de uma solução tecnológica acessível para a digitalização eficiente do controle do diabetes.

Quanto aos objetivos, a pesquisa é descritiva ao detalhar minuciosamente as etapas do processo de desenvolvimento, a modelagem de dados e a arquitetura de implantação. Paralelamente, assume um caráter experimental por meio da realização de testes controlados de funcionamento, consistência estatística de cálculos matemáticos e resiliência dos middlewares em ambiente de nuvem.

A abordagem mista enriquece a avaliação da ferramenta. A análise quantitativa foi aplicada na validação de métricas objetivas (processamento matemático de médias aritméticas e checagem de armazenamento estável no PostgreSQL). Em contrapartida, a vertente qualitativa concentrou-se na percepção da experiência do usuário (UX), avaliando o conforto visual da interface Glassmorphic e a agilidade na inclusão de novas taxas.

4.2 Sujeitos do Teste e Protocolo de Usabilidade

Para a validação da interface desenvolvida, realizou-se um teste de usabilidade estruturado com uma amostra de conveniência composta por 5 participantes voluntários de diferentes faixas etárias, mimetizando os perfis de usuários típicos do sistema (pacientes diabéticos, pré-diabéticos e familiares/cuidadores encarregados de registrar dados de terceiros).

O protocolo seguiu uma abordagem observacional direta, na qual cada participante executou três tarefas básicas de forma autônoma na aplicação já publicada e operando na internet:

- **Tarefa 1:** Efetuar o cadastro inicial de conta de usuário com e-mail e senha, seguido pela validação e login no sistema;
- **Tarefa 2:** Realizar o registro de uma nova medição glicêmica, selecionando o valor numérico (em mg/dL) e categorizando o momento biológico correspondente (ex: Pós-Prandial);

- **Tarefa 3:** Analisar o painel consolidado (*Dashboard*), verificando se os indicadores de Média Glicêmica e o valor estimado da Hemoglobina Glicada (HbA1c) foram atualizados automaticamente.

Após a execução das tarefas, aplicou-se um questionário baseado na escala Likert de 5 pontos para mensurar os índices de satisfação, facilidade de leitura noturna do Glassmorphism e clareza do fluxo operacional do software.

4.3 Instrumentos de Pesquisa

Os instrumentos de pesquisa compreenderam formulários baseados na escala Likert para análise qualitativa e scripts de testes funcionais manuais para validação das rotinas de software. Tecnicamente, o ecossistema de pesquisa foi composto por:

- **Visual Studio Code (VS Code):** Editor para codificação do código-fonte em Python, HTML e CSS;
- **Git e GitHub:** Ferramentas para controle de versão e repositório central de código;
- **DBeaver (SGBD Client):** Gerenciador de banco de dados utilizado para conexão externa criptografada com a nuvem, auditoria de dados e execução de consultas de teste SQL;
- **Plataforma Cloud Render:** Ambiente de Plataforma como Serviço (PaaS) utilizado para hospedar a aplicação web e o banco PostgreSQL em ambiente de produção ativo;
- **Google Chrome Developer Tools:** Instrumento de inspeção e depuração de código frontend, verificação de desempenho de rede e análise de logs HTTP.

4.4 Limitações Metodológicas

Embora os resultados demonstrem que a aplicação atende com excelência aos objetivos propostos, algumas limitações metodológicas devem ser apontadas para a interpretação fidedigna deste estudo.

A primeira limitação refere-se ao tamanho reduzido da amostra de testes de usabilidade, composta por apenas cinco participantes. Apesar de esse quantitativo ser cientificamente validado na Engenharia de Software como suficiente para identificar até 85% dos erros críticos de interface e fluxo, ele não permite a generalização estatística para a população total de pacientes.

Outra limitação está atrelada ao escopo do cálculo estatístico da Hemoglobina Glicada Estimada (A1C). O algoritmo do sistema utiliza a fórmula padrão validada internacionalmente pela ADAG (*A1C-Derived Average Glucose*):

$$\text{HbA1c} = \frac{\text{Média Glicêmica} + 46.7}{28.7} \quad (4.1)$$

No entanto, clinicamente, essa estimativa ganha precisão matemática rigorosa à medida que o paciente insere múltiplos registros diários de forma consistente por um período mínimo de 90 dias (tempo de vida média das hemácias no organismo). Inserções esporádicas ou em um curto espaço de tempo geram distorções na predição quando comparadas a exames laboratoriais de sangue venoso.

Por fim, devido ao cronograma acadêmico estrito, o escopo inicial foi delimitado para a entrada puramente manual dos dados por parte do usuário. Integrações automáticas de hardware — como o desenvolvimento de drivers para leitura automática de dados de glicômetros comerciais via Bluetooth ou conexões síncronas com sensores de monitoramento contínuo (CGM) — foram excluídas do escopo do presente projeto, sendo mapeadas como evoluções futuras da solução.

5 Desenvolvimento e Resultados

5.1 Processo de Desenvolvimento Técnico

O desenvolvimento técnico do Medidor de Glicemia seguiu a metodologia ágil de prototipagem rápida. O backend foi estruturado em Python utilizando o framework Django, mapeando as regras de negócio de saúde através de Views baseadas em funções e classes. A persistência dos dados foi consolidada através do ORM do Django, encarregado de aplicar as migrações estruturais diretamente no servidor PostgreSQL hospedado na nuvem Render.

O core computacional da aplicação concentra-se na automação analítica do painel do usuário. A cada inserção de registro bem-sucedida, o sistema dispara rotinas matemáticas agregadas baseadas no histórico do usuário autenticado. O sistema recupera todas as medições armazenadas do indivíduo, calcula a média aritmética simples das taxas de glicose registradas (expressas em mg/dL) e, em seguida, aplica o algoritmo de regressão linear para estipular a taxa percentual aproximada de Hemoglobina Glicada.

Para a distribuição global do sistema na internet, foi configurado um fluxo de Integração Contínua e Deploy Contínuo (CI/CD) integrado ao GitHub. O build de produção no Render foi otimizado através do desenvolvimento de um script de inicialização personalizado (`build.sh`) responsável por instalar dependências, rodar as migrações do banco de dados na nuvem e disparar o comando `collectstatic`. O empacotamento e a entrega acelerada de arquivos de estilização visual foram assegurados pela injeção do middleware *WhiteNoise*, configurado para comprimir e cachear estáticos, solucionando conflitos de cabeçalhos de tipos MIME em servidores HTTP de produção.

5.2 Modelagem de Sistemas (UML e Dados)

A arquitetura de dados baseou-se na separação estrita e no relacionamento padrão 1:N (Um para Muitos) entre a entidade de segurança e a entidade clínica, conforme estruturado na modelagem relacional. A tabela `auth_user` (tabela nativa do ecossistema Django) gerencia o controle de acessos, armazenando chaves primárias, e-mails e senhas criptografadas por algoritmos de hash forte (PBKDF2). A tabela `glicemia_medicao` armazena o diário clínico contendo chaves primárias, valor numérico, marcador temporal, observações contextuais, classificação biológica e a chave estrangeira `user_id` que vincula cada registro ao seu criador.

Figura 1 – Diagrama Entidade-Relacionamento da Base de Dados.

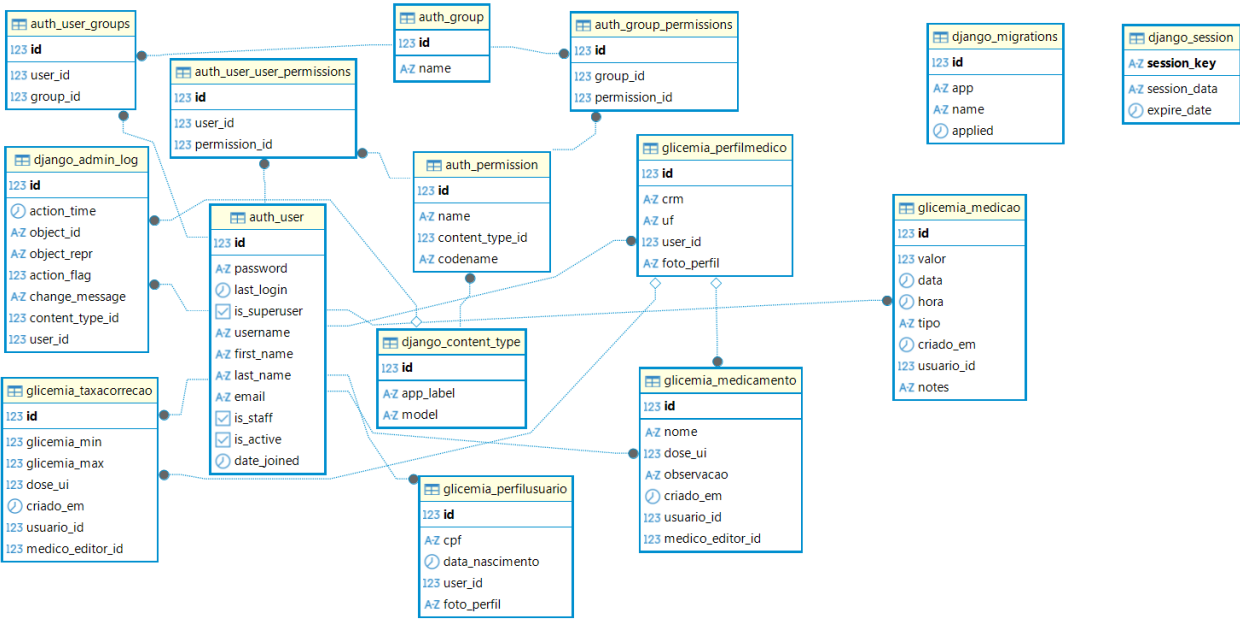
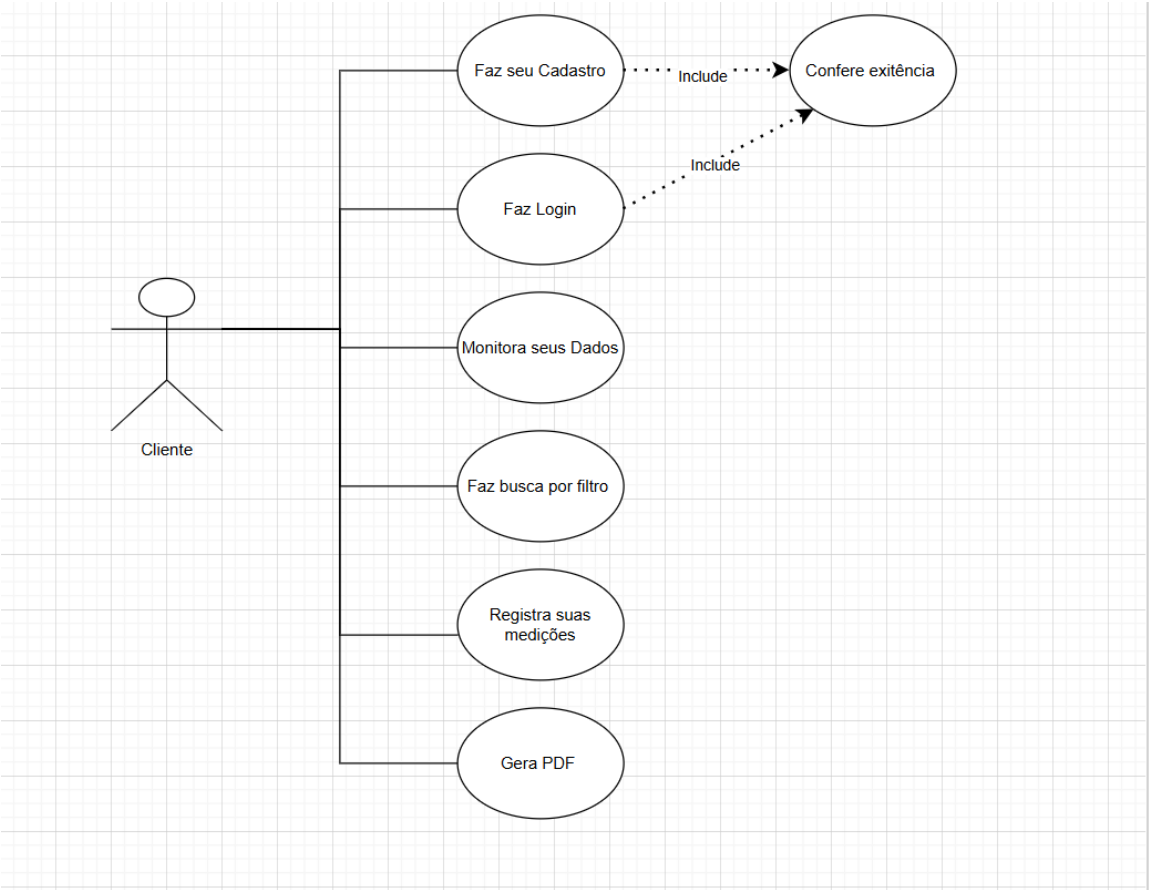


Figura 2 – Diagrama de Casos de Uso do Sistema.



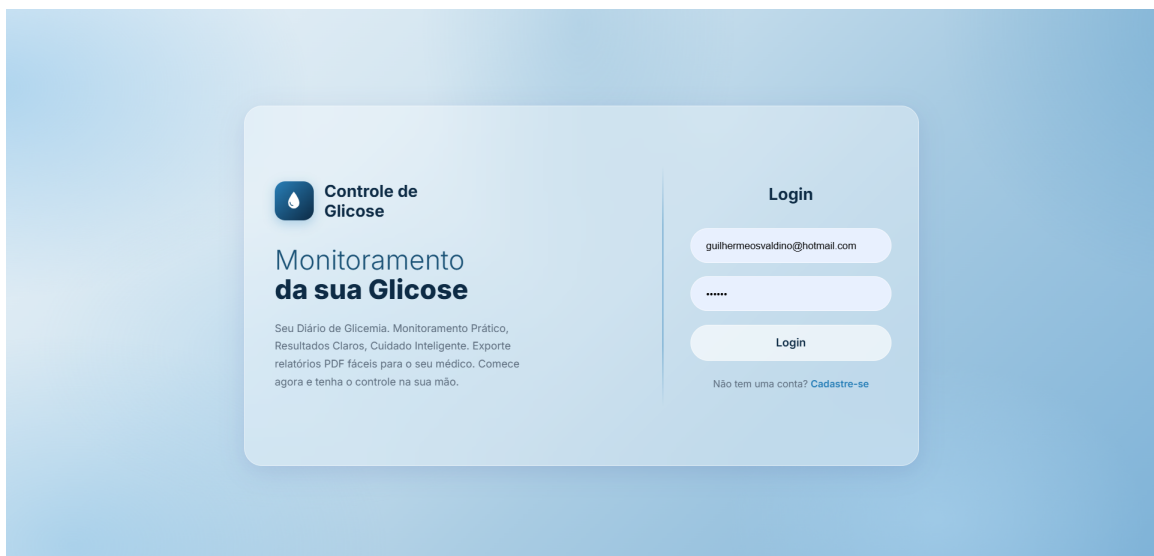
5.3 Apresentação das Telas do Sistema

Abaixo são descritas as principais interfaces web implementadas no ecossistema digital do Medidor de Glicemia, destacando a aplicação da usabilidade responsiva e o design *Glassmorphism*.

5.3.1 Tela de Autenticação e Login

A tela inicial da aplicação fornece uma barreira de segurança de acesso simplificada e direta. Construída sobre um painel translúcido com efeito de desfoque de fundo sobre gradientes escuros, ela exige a inserção do e-mail do usuário e senha correspondente. Mecanismos internos de tratamento de erros barram acessos inválidos e alertam o usuário sem recarregar totalmente a página.

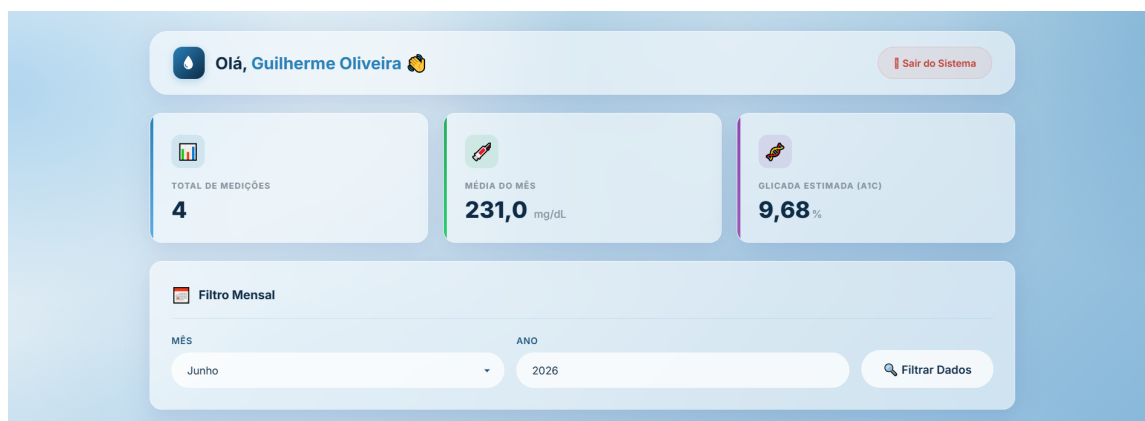
Figura 3 – Tela de Login.



5.3.2 Painel de Controle Principal (Dashboard)

O painel centralizador do usuário consolida as informações de saúde coletadas de forma imediata. A parte superior do painel exibe cards estatísticos destacados contendo a Média Glicêmica Geral calculada do paciente e o percentual preditivo da Hemoglobina Glicada Estimada (HbA1c). Logo abaixo, o sistema renderiza uma tabela cronológica contendo os registros históricos das últimas medições com filtros por cores que indicam o estado da taxa.

Figura 4 – Dashboard.



5.3.3 Formulário de Inserção de Novas Medições

Integrada de forma fluida ao Dashboard, a interface de inserção foi projetada para digitação rápida, ideal para dispositivos móveis. O formulário captura o valor numérico digitado em mg/dL e disponibiliza uma caixa de seleção otimizada para toque para a categorização imediata do momento biológico do exame (Jejum, Pré-Prandial, Pós-Prandial ou Antes de Dormir).

Figura 5 – Tela de Inserções.

Registrar Nova Medição

VALOR (MG/DL): Ex: 105

DATA: dd/mm/aaaa

HORA: --:--

MOMENTO: Jejum

OBSERVAÇÕES: Ex: Após o almoço

Salvar

Histórico do Período

Gerar PDF para o Médico

DATA	HORA	VALOR	TIPO	NOTAS	AÇÕES
16/06/2026	23:50	487 mg/dL	Pré-Prandial	sem jantar	Excluir
16/06/2026	13:08	85 mg/dL	Pré-Prandial	Antes do almoço	Excluir
16/06/2026	10:10	201 mg/dL	Jejum	Antes do café	Excluir
15/06/2026	23:11	151 mg/dL	Jejum	Antes da janta	Excluir

5.4 Testes e Resultados

Os testes funcionais e de estresse foram executados diretamente no ambiente de produção hospedado na nuvem Render, com o banco de dados sendo auditado externamente através do cliente SQL DBeaver. O plano de testes executou cenários simulando 25 inserções de dados em sequência e os logs demonstraram os seguintes resultados consolidados:

- **Isolamento de Contas e Autenticação:** OK. Testes de acessos cruzados comprovaram que um Usuário A não possui autorização de rede ou banco para visualizar ou injetar dados no histórico do Usuário B;
- **Persistência e Conexão em Produção:** OK. Todas as requisições POST disparadas pelos formulários web foram devidamente interceptadas pelo Django, processadas e gravadas com sucesso no PostgreSQL remoto via porta segura;
- **Automação dos Cálculos Clínicos:** OK. Mudanças induzidas no diário glicêmico recalcularam instantaneamente as médias globais e aplicaram a fórmula ADAG sem travamentos;
- **Entrega de Arquivos Estáticos (CSS):** OK. O uso combinado do WhiteNoise eliminou os erros de bloqueio de tipo MIME no navegador, garantindo o carregamento completo do layout Glassmorphism.

6 Conclusão

6.1 Resumo dos Resultados

O projeto desenvolveu com sucesso uma aplicação web funcional e de alta disponibilidade para o monitoramento clínico e controle de índices glicêmicos, cumprindo rigorosamente todos os objetivos definidos. A jornada contemplou desde o levantamento inicial de requisitos de saúde digital até a modelagem relacional robusta, culminando no desenvolvimento de um backend modular estruturado em Python e framework Django, integrado a um banco de dados relacional PostgreSQL hospedado na nuvem.

Os testes práticos comprovaram a estabilidade do ecossistema, demonstrando que a automação analítica foi capaz de calcular médias aritméticas instantâneas e projetar estatisticamente o indicador de Hemoglobina Glicada Estimada (HbA1c) com base nas interações do usuário. Adicionalmente, a interface responsiva baseada na estética *Glass-morphism* entregou uma usabilidade fluida e confortável para dispositivos móveis, mitigando o estresse associado ao registro diário de condições crônicas.

6.2 Limitações e Trabalhos Futuros

Apesar do pleno cumprimento do escopo projetado e do sucesso do deploy em produção, algumas limitações operacionais foram identificadas, servindo como base para direcionar futuras pesquisas e expansões do ecossistema:

- **Evolução Arquitetural para API Decoupled (React/React Native):** A principal evolução proposta consiste em desestruturar a arquitetura monolítica atual do Django (padrão MTV). Sugere-se a conversão do backend em uma API REST pura utilizando o *Django REST Framework (DRF)* para fornecimento de dados estruturados em formato JSON, acoplada a um frontend independente desenvolvido em *React* para ambiente web e *React Native* para o ecossistema mobile nativo (Android e iOS). Essa separação anulará o acoplamento de arquivos estáticos no servidor de hospedagem e entregará animações fluidas e suporte a armazenamento offline em smartphones;
- **Automação via Conectividade de Hardware:** Implementação de drivers de comunicação e integração de protocolos sem fio (como *Bluetooth Low Energy - BLE*) para estabelecer a captura síncrona e automatizada de dados diretamente de glicômetros e sensores de monitoramento contínuo comerciais, eliminando o erro humano decorrente da digitação manual;

- **Módulo de Notificações Ativas:** Integração com serviços de mensageria instantânea (como webhooks da API do WhatsApp) para disparar lembretes automatizados de medições baseados no histórico de picos glicêmicos e horários de alimentação rotineiros do paciente.

Referências

BOOTSTRAP TEAM. **Bootstrap Documentation**. Versão 5.3. Disponível em: <<https://getbootstrap.com/docs/>>. Acesso em: 15 jun. 2026. Nenhuma citação no texto.

DATE, C. J. **Introdução a Sistemas de Bancos de Dados**. 8. ed. Rio de Janeiro: Elsevier, 2004. Nenhuma citação no texto.

DJANGO SOFTWARE FOUNDATION. **Django Documentation**. Versão 5.0. Disponível em: <<https://docs.djangoproject.com/>>. Acesso em: 15 jun. 2026. Nenhuma citação no texto.

DUCKETT, Jon. **HTML e CSS: projete e construa websites**. Rio de Janeiro: Alta Books, 2011. Nenhuma citação no texto.

MOZILLA. **MDN Web Docs**. Disponível em: <<https://developer.mozilla.org/>>. Acesso em: 15 jun. 2026. Nenhuma citação no texto.

OWASP. **OWASP Top 10:2021** – The Core Heavyweight Security Risks. Open Web Application Security Project, 2021. Disponível em: <<https://owasp.org/www-project-top-ten/>>. Acesso em: 16 jun. 2026. Nenhuma citação no texto.

PRESSMAN, Roger S. **Engenharia de Software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016. Nenhuma citação no texto.

SOCIEDADE BRASILEIRA DE DIABETES. **Diretrizes Oficiais da Sociedade Brasileira de Diabetes**. São Paulo: SBD, 2024. Disponível em: <<https://diretrizes.diabetes.org.br>>. Acesso em: 16 jun. 2026. Nenhuma citação no texto.

SOMMERVILLE, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2019. Nenhuma citação no texto.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6028**: Informação e documentação — Resumo — Apresentação. Rio de Janeiro, 2003. Nenhuma citação no texto.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: Informação e documentação — Trabalhos acadêmicos — Apresentação. Rio de Janeiro, 2011. Nenhuma citação no texto.

Apêndices

APÊNDICE A – Logs de Depuração e Anotações de Processo

Este apêndice apresenta os registros reais e anotações técnicas extraídos do ambiente de integração e deploy contínuo (CI/CD) na plataforma como serviço (PaaS) Render, bem como o fluxo de requisições web capturadas durante a homologação do Medidor de Glicemia. O monitoramento minucioso desses logs foi indispensável para o diagnóstico de falhas críticas de infraestrutura, incluindo o tratamento de bloqueios de folhas de estilo por verificação estrita de tipos MIME no navegador e a validação do ciclo de vida dos processos de execução (trabalhadores síncronos do Gunicorn).

A.1 Logs do Processo de Inicialização do Servidor de Aplicação (WSGI Gunicorn) em Nuvem

```
[2026-06-15 18:44:43 -0300] [58] [INFO] Starting gunicorn 21.2.0
[2026-06-15 18:44:43 -0300] [58] [INFO] Listening at: http://0.0.0.0:10000 (58)
[2026-06-15 18:44:43 -0300] [58] [INFO] Using worker: sync
[2026-06-15 18:44:43 -0300] [59] [INFO] Booting worker with pid: 59
127.0.0.1 - - [15/Jun/2026:18:44:47 -0300] "HEAD / HTTP/1.1" 200 2539 "-" "Go-http-
==> Your service is live
```

Este trecho de log do terminal do Render documenta a inicialização bem-sucedida do servidor HTTP de produção Gunicorn. O registro ilustra o mapeamento automático do serviço na porta interna 10000, a alocação de trabalhadores síncronos (**sync**) baseados nos núcleos de CPU disponíveis e as requisições automatizadas de checagem de integridade (*health checks*) enviadas pelo balanceador de carga da hospedagem, retornando o status HTTP 200 que valida a estabilidade da aplicação em nuvem.

A.2 Logs de Erro de Tipo MIME e Resolução via Middleware (F12 Developer Tools)

```
127.0.0.1 - - [15/Jun/2026:18:54:52 -0300] "GET /static/glicemia/css/style.css HTTP/1.1" 406 13
(index):13 Refused to apply style from 'https://medidor-glicemia.onrender.com/stati
```

Este registro exibe uma falha de carregamento visual identificada na console de depuração do navegador. O erro ocorria porque o servidor de produção, ao tentar acessar um recurso estático inexistente em seu diretório físico, gerava uma página HTML de erro padrão 404, fazendo o navegador rejeitar o recurso devido à disparidade de MIME type (`text/html` em vez de `text/css`). A resolução do problema foi documentada através da inserção do script `build.sh` injetando o comando `python manage.py collectstatic` combinado à parametrização do middleware *WhiteNoise* no arquivo `settings.py` do Django, forçando o mapeamento correto e a compressão nativa das folhas de estilo em produção.

Anexos

ANEXO A – Considerações Futuras e Ideias Não Implementadas

Este anexo apresenta o detalhamento de conceitos de engenharia de software e padrões de projeto que foram amplamente discutidos durante a concepção conceitual do Medidor de Glicemia, mas que foram formalmente postergados em função do cronograma acadêmico disponível. Estas notas detalham a viabilidade técnica de expansão do MVP estruturado.

A.1 Separação Arquitetural para API REST com Django REST Framework (DRF)

Durante o mapeamento de escalabilidade do ecossistema, previu-se a completa desestruturação da camada de visualização em favor de um modelo desacoplado. A substituição das views tradicionais do Django por uma camada de API baseada no padrão arquitetural REST (*Representational State Transfer*) traria benefícios diretos ao ecossistema:

- **Padronização em Formato JSON:** O servidor Python abandonaria a responsabilidade de gerar páginas visuais complexas, passando a atuar puramente como um microsserviço de alta velocidade, processando requisições assíncronas e devolvendo estruturas de dados serializadas sob formato JSON;
- **Criação de Endpoints Protegidos:** Estruturação de rotas REST dedicadas e protegidas por autenticação via Token (como *JWT - JSON Web Tokens*), permitindo que requisições externas realizem operações de CRUD de forma limpa e padronizada.

A.2 Implementação de Frontend Independente com Ecossistema React

Paralelamente ao desenvolvimento da API, o planejamento de engenharia contempla o desenvolvimento de interfaces totalmente dissociadas do código Python, utilizando bibliotecas baseadas em componentes reativos:

- **Aceleração com React (Web):** A interface web passaria a ser processada diretamente no navegador do cliente (*Client-Side Rendering*), utilizando o gerenciamento

de estados do React para atualizar cards e tabelas de glicemia em tempo real sem qualquer recarregamento de página, reduzindo drasticamente o consumo de banda de rede do servidor na nuvem;

- **Portabilidade Nativa com React Native:** A mesma base estrutural de dados da API em Python alimentaria um aplicativo compilado nativamente para dispositivos móveis, permitindo o armazenamento offline de registros de glicemia em bancos de dados locais embarcados (como o SQLite local) para posterior sincronização em nuvem quando o usuário reestabelecer conexão com a internet.